인지 에이전트 재구현 가이드

Unity3D C# 기반 cogARCH 구현

논문: Beyond the Shortest-Path: Towards Cognitive Occupancy Modeling in BIM

Gath-Morad et al. (2022) | Automation in Construction 135

개요

이 문서는 ETH 취리히 연구팀이 개발한 인지 에이전트 시뮬레이션 프레임워크 cogARCH를 Unity3D C#으로 재구현하는 방법을 단계별로 설명합니다. cogARCH 소스코드는 공개되어 있지 않으므로, 논문에 기술된 구조를 설계도 삼아 처음부터 구현합니다.

Unity의 Physics.Raycast, NavMeshAgent, NavMesh.SamplePosition이 수학의 대부분을 대신 처리해주기 때문에, 직접 구현 대비 코드량이 약 1/3로 줄어듭니다.

핵심 구현 구조

cogARCH는 두 축으로 나뉩니다: 환경 표현 레이어와 에이전트 인지 모듈입니다.

```
[BIM IFC 파일]
      ↓ 파싱
┌──────────────────────────────┐
│   계층적 내비게이션 공간      │
│  1. NavMesh (이동 가능 공간) │
│  2. 3D FOV (시야 품질)       │
│  3. 의미론적 공간 태그       │
│  4. 문/연결 그래프           │
└──────────────────────────────┘
      ↓ 에이전트 인식
┌──────────────────────────────┐
│   인지 에이전트 모듈         │
│  - Physics.Raycast 시야      │
│  - 메모리 (방문 공간 기억)   │
│  - 배경지식 (목적지 상식)    │
│  - 전략 선택 (4가지)         │
│  - NavMeshAgent 이동         │
└──────────────────────────────┘
```

개발 일정 요약

주차	목표	난이도
1주차	IFC import + NavMesh + ZoneTag	⭐⭐
2주차	VisionSystem (Raycast 시야)	⭐⭐⭐
3주차	MemorySystem + BackgroundKnowledge	⭐⭐
4주차	Strategy 로직 + SimulationManager	⭐⭐⭐
5~6주차	대량 시뮬레이션 + CSV 수집 + 논문 비교	⭐⭐

Step 1 — IFC → Unity 건물 Import

가장 빠른 방법은 Python 변환을 거치거나, Tridify 플러그인(무료 플랜)을 사용하는 것입니다. 논문 팀도 Tridify를 사용했습니다.

방법 A: Tridify 플러그인 (권장)

- Unity Asset Store 에서 Tridify 설치
- IFC 파일 → Tridify 클라우드 업로드
- Unity 내에서 변환된 오브젝트 자동 생성
- IFC 클래스(IfcSpace, IfcDoor 등) 메타데이터 유지

방법 B: Python + Blender 변환

```
# 터미널에서 실행
pip install ifcopenshell
# Blender에서: File > Import > IFC
# 이후: File > Export > FBX → Unity Assets 폴더에 드래그
```

방법 C: BIM Interoperability Tools (Autodesk)

Revit 사용자라면 BIM Interoperability Tools로 IFC → Revit → FBX 변환 가능.

> *논문의 Zollverein School IFC 데이터는 github.com/MichalGath/DataWayfindingExperimentSimulationZollverein 에서 확인 가능. 저장소 접속 후 직접 확인 필요.*

Step 2 — 공간 태그 시스템 (ZoneTag)

각 방에 의미 정보를 붙입니다. Unity Inspector에서 드래그 앤 드롭으로 설정 가능하며, NavMesh는 클릭 몇 번으로 자동 생성됩니다.

ZoneTag.cs

```
// ZoneTag.cs — 각 방 오브젝트에 부착
public class ZoneTag : MonoBehaviour
{
    public enum ZoneType
    {
        Auditorium, Cafeteria, Office,
        StudyArea, Patio, RoofTerrace,
        Corridor, Stairs, Elevator
    }

    public ZoneType zoneType;
    public int floorNumber;        // 몇 층인지
    public bool isPublic = true;   // 공공 공간 여부
    public float integrationValue; // Space Syntax 중심성 (선택)

    void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("CognitiveAgent"))
            other.GetComponent<CognitiveAgent>()
                 .OnEnterZone(this);
    }
}
```

NavMesh 자동 생성

```
Window > AI > Navigation
→ 건물 전체 선택 > Navigation Static 체크
→ Bake 버튼 클릭
→ 완료 (계단 포함 자동 처리)
```

> *각 방의 Collider를 Trigger로 설정해두면 에이전트가 해당 공간에 진입했는지 자동 감지됩니다.*

Step 3 — 3D 시야 (VisionSystem)

논문의 핵심 차별점입니다. Unity의 Physics.Raycast가 3D 시야 계산을 대신 해줍니다. 직접 수학할 필요가 없으며, 유리벽과 아트리움 투과 처리, 층간 시야가 자동으로 처리됩니다.

VisionSystem.cs

```
// VisionSystem.cs
public class VisionSystem : MonoBehaviour
{
    [Header("시야 설정")]
    public float maxViewDistance = 15f;
    public float horizontalFOV   = 120f;  // 수평 시야각
    public float verticalFOV     = 60f;   // 수직 시야각 (층간 감지용)
    public int   rayCountH       = 24;    // 수평 광선 수
    public int   rayCountV       = 8;     // 수직 광선 수

    // 유리 레이어 — 이 레이어는 광선이 통과
    public LayerMask glassMask;
    public LayerMask solidMask;

    public List<ZoneTag> GetVisibleZones()
    {
        var visible = new HashSet<ZoneTag>();

        float hStep = horizontalFOV / rayCountH;
        float vStep = verticalFOV   / rayCountV;

        for (int h = 0; h < rayCountH; h++)
        for (int v = 0; v < rayCountV; v++)
        {
            float yaw   = -horizontalFOV/2 + h * hStep;
            float pitch = -verticalFOV/2   + v * vStep;

            Vector3 dir = Quaternion.Euler(pitch, yaw, 0)
                          * transform.forward;

            // 1차: 불투명 벽 체크
            if (Physics.Raycast(transform.position, dir,
                out RaycastHit hit, maxViewDistance, solidMask))
            {
                var zone = hit.collider.GetComponentInParent<ZoneTag>();
                if (zone != null) visible.Add(zone);
            }
            // 2차: 유리 투과 (아트리움/Glass 시나리오 핵심)
            else if (Physics.Raycast(transform.position, dir,
                out hit, maxViewDistance, glassMask))
            {
                Vector3 behindGlass = hit.point + dir * 0.1f;
                if (Physics.Raycast(behindGlass, dir,
                    out RaycastHit hit2, maxViewDistance, solidMask))
                {
                    var zone = hit2.collider.GetComponentInParent<ZoneTag>();
                    if (zone != null) visible.Add(zone);
                }
            }
        }
        return visible.ToList();
    }

    // 특정 목적지가 보이는지 빠르게 확인
    public bool CanSeeZone(ZoneTag target)
```

```
    {
        Vector3 dirToTarget = (target.transform.position
                               - transform.position).normalized;
        float dist = Vector3.Distance(transform.position,
                                      target.transform.position);
        return !Physics.Raycast(transform.position, dirToTarget,
                                dist, solidMask);
    }
}
```

> *Unity Layer 설정에서 유리 오브젝트를 'Glass' 레이어에 배치하고, glassMask에 할당하면 투과 처리가 자동으로 됩니다.*

Step 4 — 메모리 시스템 (MemorySystem)

에이전트가 방문하거나 시야에 들어온 공간을 기억하고, 시간이 지나면 기억이 희미해지는 decay 시스템입니다.

MemorySystem.cs

```
// MemorySystem.cs
[System.Serializable]
public class MemoryRecord
{
    public ZoneTag   zone;
    public Vector3   position;
    public float     strength;    // 1.0 = 생생함, 0.0 = 완전히 잊음
    public float     lastSeenTime;
}

public class MemorySystem : MonoBehaviour
{
    public float decayRate = 0.05f; // 초당 감소량

    private Dictionary<ZoneTag, MemoryRecord> _memory
        = new Dictionary<ZoneTag, MemoryRecord>();

    // 시야에 들어온 공간들 기억에 추가
    public void UpdateMemory(List<ZoneTag> visibleZones)
    {
        foreach (var zone in visibleZones)
        {
            if (_memory.ContainsKey(zone))
            {
                _memory[zone].strength     = 1.0f;
                _memory[zone].lastSeenTime = Time.time;
            }
            else
            {
                _memory[zone] = new MemoryRecord
                {
                    zone         = zone,
                    position     = zone.transform.position,
                    strength     = 1.0f,
                    lastSeenTime = Time.time
                };
            }
        }
    }

    void Update()
    {
        // 매 프레임 기억 decay
        foreach (var record in _memory.Values)
            record.strength = Mathf.Max(0,
                record.strength - decayRate * Time.deltaTime);
    }

    // "계단 어디 있었지?" → 가장 강한 기억 반환
    public MemoryRecord RecallByType(ZoneTag.ZoneType type)
    {
        MemoryRecord best = null;
        foreach (var record in _memory.Values)
        {
            if (record.zone.zoneType == type && record.strength > 0.1f)
                if (best == null || record.strength > best.strength)
```

```
                    best = record;
        }
        return best;
    }

    public IEnumerable<MemoryRecord> GetAllRecords()
        => _memory.Values;
}
```

Step 5 — 배경지식 + 전략 선택 + 이동 (CognitiveAgent)

모든 컴포넌트를 합친 메인 에이전트 스크립트입니다. 논문에 기술된 4가지 전략(Floor, Direction, Central, Perimeter)을 구현합니다.

전략 설명

전략	발동 조건	행동
Floor	목적지가 다른 층에 있을 것으로 예상	가장 가까운 계단/엘리베이터로 이동
Direction	기억에 목적지 단서가 있음	기억 속 위치 방향으로 이동
Central	기억에 단서 없음 + 공공장소 목적지	시각적으로 통합된 공간으로 이동
Perimeter	기억에 단서 없음 + 비공공 목적지	벽을 따라 탐색

CognitiveAgent.cs

```
// CognitiveAgent.cs
[RequireComponent(typeof(NavMeshAgent))]
[RequireComponent(typeof(VisionSystem))]
[RequireComponent(typeof(MemorySystem))]
public class CognitiveAgent : MonoBehaviour
{
    public ZoneTag.ZoneType destination;
    public float perceptionInterval = 0.5f; // 0.5초마다 지각

    private NavMeshAgent _nav;
    private VisionSystem _vision;
    private MemorySystem _memory;

    // 배경지식 테이블 (논문 [53] 기반)
    private static readonly Dictionary<ZoneTag.ZoneType, (bool isPublic, bool
highFloor)>
    BackgroundKnowledge = new()
    {
        [ZoneTag.ZoneType.Auditorium]  = (isPublic: true,  highFloor: false),
        [ZoneTag.ZoneType.Cafeteria]   = (isPublic: true,  highFloor: false),
        [ZoneTag.ZoneType.Office]      = (isPublic: false, highFloor: false),
        [ZoneTag.ZoneType.RoofTerrace] = (isPublic: true,  highFloor: true),
        [ZoneTag.ZoneType.Patio]       = (isPublic: true,  highFloor: false),
        [ZoneTag.ZoneType.StudyArea]   = (isPublic: true,  highFloor: false),
    };

    enum Strategy { SeekVisible, Floor, Direction, Central, Perimeter }
    Strategy _currentStrategy;

    void Start()
    {
        _nav    = GetComponent<NavMeshAgent>();
        _vision = GetComponent<VisionSystem>();
        _memory = GetComponent<MemorySystem>();
        InvokeRepeating(nameof(PerceptionCycle), 0f, perceptionInterval);
    }
```

```
void PerceptionCycle()
{
    // 1. 지각
    var visible = _vision.GetVisibleZones();
    _memory.UpdateMemory(visible);

    // 2. 목적지 직접 보이면 바로 이동
    foreach (var zone in visible)
    {
        if (zone.zoneType == destination)
        {
            _nav.SetDestination(zone.transform.position);
            _currentStrategy = Strategy.SeekVisible;
            return;
        }
    }

    // 3. 전략 선택 및 실행
    SelectStrategy();
    ExecuteStrategy();
}

void SelectStrategy()
{
    var kb = BackgroundKnowledge[destination];
    int targetFloor = kb.highFloor ? 4 : 1;
    int myFloor = GetCurrentFloor();

    if (Mathf.Abs(targetFloor - myFloor) > 0)
    {
        _currentStrategy = Strategy.Floor;
        return;
    }

    var recall = _memory.RecallByType(destination);
    if (recall != null)
    {
        _currentStrategy = Strategy.Direction;
        return;
    }

    _currentStrategy = kb.isPublic
        ? Strategy.Central
        : Strategy.Perimeter;
}

void ExecuteStrategy()
{
    switch (_currentStrategy)
    {
        case Strategy.Floor:
            MoveToNearestStairs();
            break;
        case Strategy.Direction:
            var recall = _memory.RecallByType(destination);
            _nav.SetDestination(recall.position);
            break;
        case Strategy.Central:
            MoveToMostVisibleArea();
            break;
        case Strategy.Perimeter:
            MoveAlongPerimeter();
            break;
    }
}
```

```
    void MoveToNearestStairs()
    {
        var stairMemory = _memory.RecallByType(ZoneTag.ZoneType.Stairs);
        if (stairMemory != null)
        {
            _nav.SetDestination(stairMemory.position);
            return;
        }
        var stairs = FindObjectsByType<ZoneTag>(FindObjectsSortMode.None);
        ZoneTag nearest = null;
        float minDist = float.MaxValue;
        foreach (var z in stairs)
        {
            if (z.zoneType != ZoneTag.ZoneType.Stairs) continue;
            float d = Vector3.Distance(transform.position,
z.transform.position);
            if (d < minDist) { minDist = d; nearest = z; }
        }
        if (nearest != null)
            _nav.SetDestination(nearest.transform.position);
    }

    void MoveToMostVisibleArea()
    {
        MemoryRecord best = null;
        foreach (var r in _memory.GetAllRecords())
            if (r.zone.isPublic && (best == null || r.strength > best.strength))
                best = r;
        if (best != null)
            _nav.SetDestination(best.position);
    }

    void MoveAlongPerimeter()
    {
        Vector3 perimeterTarget = transform.position + transform.right * 5f;
        if (NavMesh.SamplePosition(perimeterTarget,
            out NavMeshHit hit, 3f, NavMesh.AllAreas))
            _nav.SetDestination(hit.position);
    }

    int GetCurrentFloor()
        => Mathf.RoundToInt(transform.position.y / 3.5f);

    public List<Vector3> trajectory = new();
    void LateUpdate()
    {
        if (Time.frameCount % 12 == 0)
            trajectory.Add(transform.position);
    }

    public void OnEnterZone(ZoneTag zone)
    {
        if (zone.zoneType == destination)
        {
            Debug.Log($"도착! 이동거리: {CalculateTotalDistance()}m");
            CancelInvoke(nameof(PerceptionCycle));
        }
    }

    float CalculateTotalDistance()
    {
        float total = 0;
        for (int i = 1; i < trajectory.Count; i++)
            total += Vector3.Distance(trajectory[i-1], trajectory[i]);
        return total;
```

```
    }
}
```

Step 6 — 시뮬레이션 실행 및 데이터 수집

논문처럼 대량 시뮬레이션을 돌리고 CSV로 내보내는 관리자 스크립트입니다.

SimulationManager.cs

```
// SimulationManager.cs
public class SimulationManager : MonoBehaviour
{
    public GameObject agentPrefab;
    public Transform  spawnPoint;
    public int        runsPerDestination = 100;

    private List<SimResult> _results = new();

    [System.Serializable]
    struct SimResult
    {
        public ZoneTag.ZoneType destination;
        public float            distanceCovered;
        public float            timeTaken;
        public string           scenario;
    }

    IEnumerator RunAllSimulations()
    {
        foreach (ZoneTag.ZoneType dest in
                 System.Enum.GetValues(typeof(ZoneTag.ZoneType)))
        {
            for (int i = 0; i < runsPerDestination; i++)
            {
                var go = Instantiate(agentPrefab, spawnPoint.position,
                                     Random.rotation);
                var agent = go.GetComponent<CognitiveAgent>();
                agent.destination = dest;

                yield return new WaitForSeconds(30f); // 타임아웃
                Destroy(go);
            }
        }
        ExportCSV();
    }

    void ExportCSV()
    {
        var sb = new System.Text.StringBuilder();
        sb.AppendLine("scenario,destination,distance,time");
        foreach (var r in _results)
            sb.AppendLine($"{r.scenario},{r.destination}," +
                          $"{r.distanceCovered},{r.timeTaken}");
        System.IO.File.WriteAllText("simulation_results.csv", sb.ToString());
        Debug.Log("CSV 저장 완료");
    }
}
```

Step 7 — 논문 데이터와 비교 검증

논문의 GitHub 저장소 데이터와 내 시뮬레이션 결과를 Python으로 비교합니다.

검증 스크립트 (Python)

```
import pandas as pd
import scipy.stats as stats
from statsmodels.formula.api import mixedlm

my_results    = pd.read_csv("simulation_results.csv")
paper_results = pd.read_csv("cognitive_agents_paper.csv")

# 논문 Table 2의 LMEMR 재현
model = mixedlm("distance ~ scenario", my_results,
                groups=my_results["destination"])
result = model.fit()
print(result.summary())
# → 논문의 -110.328m와 내 값 비교

# KDE 경로 밀도 시각화
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import gaussian_kde

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for i, scenario in enumerate(["Base", "Atria", "Glass"]):
    data = my_results[my_results.scenario == scenario]
    axes[i].set_title(f"Scenario: {scenario}")
plt.savefig("kde_comparison.png", dpi=150)
print("시각화 저장 완료")
```

주요 비교 지표

지표	논문 인지 에이전트 값	목표 범위
Base → Glass 거리 변화	-110.328m	-80m ~ -130m
Base → Atria 거리 변화	-87.290m	-60m ~ -110m
ROC-AUC (Glass)	~0.85	> 0.70
F1 Score (Glass)	~0.80	> 0.65

> *완전히 동일한 수치를 목표로 하기보다, 인지 에이전트가 최단경로 에이전트보다 사람 행동에 더 가까운지를 확인하는 것이 핵심입니다.*

점진적 구현 전략 (추천)

처음부터 완전 구현보다 기능을 단계별로 추가하는 방식이 현실적입니다. 각 버전에서 논문 데이터와 비교 검증이 가능합니다.

버전	구현 내용	검증 가능 항목
v0.1	A* 최단경로 에이전트 (NavMeshAgent만)	논문의 최단경로 에이전트 한계 재현
v0.2	2D 시야 + 기본 전략 추가	건물별 경로 차이 발생 여부
v0.3	3D 시야 + 층간 시야 추가	Glass/Atria 시나리오 효과 감지
v0.4	메모리 decay + 배경지식 추가	전략 선택 다양성
v1.0	대량 시뮬레이션 + 통계 분석	논문 Table 2 수치 비교

참고 자료

- 논문 원문: Gath-Morad et al. (2022), Automation in Construction 135, 104131
- 논문 데이터셋: github.com/MichalGath/DataWayfindingExperimentSimulationZollverein
- cogARCH 원본 논문: SimAUD 2020 - simaud.org/2020/proceedings/10.pdf
- IfcOpenShell: ifcopenshell.org
- Unity NavMesh: docs.unity3d.com/Manual/NavMesh-BuildingComponents.html
- Tridify: tridify.com

이 문서는 비전공자의 실험 재현을 위해 작성되었습니다. cogARCH 소프트웨어는 ETH 취리히 연구팀의 지적 재산이며, 저자(michal.gath@gess.ethz.ch)에게 협업을 요청하는 것도 좋은 방법입니다.